



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Castro Blanco Manuel David

Fecha:
09 de Junio 2026

Tabla de contenido

INTRODUCCIÓN.....	3
INVESTIGACIÓN.....	3
MVC (MODEL-VIEW-CONTROLLER).....	3
ARQUITECTURA EN CAPAS (LAYERED ARCHITECTURE)	4
MICROSERVICIOS	4
MONOLÍTICA	5
CLIENTE-SERVIDOR	6
CUADRO COMPARATIVO.....	7
ANÁLISIS.....	7
CASO PRÁCTICO	9
SISTEMA SELECCIONADO: ECOMMERCE (TIENDA EN LÍNEA)	9
CONCLUSIÓN	9

Introducción

Conforme los sistemas de software crecen y se vuelven más complejos, se necesita organizar el código de manera ordenada, separar responsabilidades y facilitar su mantenimiento. Los patrones de arquitectura de software son soluciones probadas a problemas que se repiten en el desarrollo y ofrecen una forma estructurada de construir aplicaciones.

El uso de un patrón adecuado mejora aspectos como la escalabilidad, la mantenibilidad, la modularidad, la reutilización de componentes y la organización general del sistema. Por el contrario, elegir mal la arquitectura puede ocasionar problemas de rendimiento, costos elevados y dificultades para crecer. En este trabajo se investigan y comparan cinco patrones: MVC, Arquitectura en Capas, Microservicios, Monolítica y Cliente-Servidor.

Investigación

MVC (Model-View-Controller)

Definición: Patrón que divide una aplicación en tres componentes: el Modelo (datos y lógica de negocio), la Vista (interfaz que ve el usuario) y el Controlador (intermediario que procesa las peticiones).

Estructura general: Tres componentes lógicos interconectados: el usuario actúa sobre la vista, el controlador procesa la acción y el modelo gestiona los datos.

Componentes principales: Modelo, Vista y Controlador.

Forma de funcionamiento: El usuario interactúa con la vista; el controlador recibe la acción, consulta o actualiza el modelo y devuelve una vista actualizada al usuario.

Ventajas: Separación clara de responsabilidades, código ordenado, mantenimiento más sencillo y trabajo en equipo más eficiente.

Desventajas: Tiene curva de aprendizaje, puede resultar excesivo para proyectos muy pequeños y volverse complejo en aplicaciones muy grandes.

Casos de uso: Aplicaciones web y de escritorio. Frameworks como Laravel, Spring MVC, ASP.NET MVC y Django lo utilizan.

Arquitectura en Capas (Layered Architecture)

Definición: Organiza el sistema en capas horizontales, cada una con una responsabilidad específica, donde cada capa se comunica únicamente con la capa contigua.

Estructura general: Normalmente se divide en capa de presentación, capa de lógica de negocio y capa de acceso a datos.

Componentes principales: Capa de presentación, capa de negocio y capa de datos (a veces se añade una capa de servicios).

Forma de funcionamiento: Las peticiones bajan de la capa superior a la inferior y las respuestas regresan hacia arriba; cada capa depende solo de la que tiene debajo.

Ventajas: Modularidad, separación clara de funciones, facilidad de mantenimiento y de pruebas, y reutilización de capas.

Desventajas: Puede generar sobrecarga al pasar entre capas, resultar rígida y perder eficiencia si hay demasiadas capas.

Casos de uso: Sistemas empresariales, aplicaciones de gestión y software corporativo.

Microservicios

Definición: Arquitectura que divide la aplicación en servicios pequeños, independientes y especializados que se comunican entre sí, normalmente mediante APIs.

Estructura general: Conjunto de servicios autónomos, cada uno con su propia lógica y, a menudo, su propia base de datos, coordinados a través de la red.

Componentes principales: Servicios independientes, API Gateway, mecanismos de comunicación (REST o mensajería) y bases de datos por servicio.

Forma de funcionamiento: Cada servicio se desarrolla, despliega y escala por separado, y se comunica con los demás mediante llamadas a través de la red.

Ventajas: Alta escalabilidad, despliegue independiente, flexibilidad tecnológica y mayor tolerancia a fallos.

Desventajas: Mayor complejidad, gestión difícil, necesidad de buena infraestructura y monitoreo, además de latencia de red.

Casos de uso: Sistemas grandes y de alto tráfico como Netflix, Amazon o Uber.

Monolítica

Definición: Toda la aplicación se construye y despliega como una sola unidad, con todos los módulos integrados en un mismo programa.

Estructura general: Un único bloque que contiene la interfaz, la lógica de negocio y el acceso a datos.

Componentes principales: Un solo proyecto o ejecutable que reúne todos los módulos internos.

Forma de funcionamiento: Todo se ejecuta dentro de un mismo proceso; cualquier cambio obliga a recompilar y desplegar la aplicación completa.

Ventajas: Simple de desarrollar, probar y desplegar al inicio, con buen rendimiento porque no hay latencia de red.

Desventajas: Difícil de escalar y mantener cuando crece; un error puede afectar a todo el sistema.

Casos de uso: Proyectos pequeños o medianos, prototipos, MVP y equipos reducidos.

Cliente-Servidor

Definición: Modelo en el que los clientes solicitan servicios o recursos y un servidor los provee, separando claramente ambos roles.

Estructura general: Uno o varios clientes conectados a uno o varios servidores a través de una red.

Componentes principales: Cliente (solicita), Servidor (responde) y la red de comunicación que los une.

Forma de funcionamiento: El cliente envía una petición, el servidor la procesa y devuelve una respuesta.

Ventajas: Centraliza los datos y el control, facilita la administración y la seguridad, y permite compartir recursos.

Desventajas: El servidor puede convertirse en un punto único de fallo y saturarse; además depende de la red.

Casos de uso: Aplicaciones web, correo electrónico, bases de datos y servicios en línea.

Cuadro comparativo

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios
Objetivo principal	Separar datos, presentación y control.	Organizar el sistema en niveles con funciones definidas.	Construir toda la app como una sola unidad.	Dividir la app en servicios pequeños e independientes.
Estructura	Modelo, Vista y Controlador.	Presentación, negocio y datos.	Un solo bloque con todo integrado.	Servicios conectados por red/API.
Organización del código	Por responsabilidad (3 componentes).	Por capas horizontales.	Todo junto en un mismo proyecto.	Separado por servicio o dominio.
Escalabilidad	Media.	Media.	Baja, difícil de escalar.	Alta, escala por servicio.
Mantenimiento	Bueno, código ordenado.	Bueno, cambios aislados por capa.	Difícil cuando el sistema crece.	Bueno por servicio, complejo en conjunto.
Complejidad	Baja a media.	Media.	Baja al inicio, alta al crecer.	Alta.
Facilidad de implementación	Fácil, los frameworks lo soportan.	Media.	Muy fácil al inicio.	Difícil.
Rendimiento	Bueno.	Bueno, con algo de sobrecarga.	Alto, todo es local.	Bueno, pero con latencia de red.
Reutilización	Media a alta.	Alta, capas reutilizables.	Baja.	Alta, servicios reutilizables.
Casos de uso recomendados	Aplicaciones web y de escritorio.	Sistemas empresariales.	Proyectos pequeños o MVP.	Sistemas grandes y de alto tráfico.
Ventajas	Orden, separación clara, fácil mantenimiento.	Modular, ordenada y fácil de probar.	Simple y rápida de desarrollar y desplegar.	Escalable, flexible y despliegue independiente.
Desventajas	Puede complicarse en apps muy grandes.	Sobrecarga y rigidez entre capas.	Difícil de escalar y mantener al crecer.	Compleja de gestionar y desplegar.

Análisis

1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

El patrón MVC, porque ofrece un buen equilibrio entre orden y simplicidad. Está soportado por la mayoría de los frameworks web, lo que acelera el desarrollo, y permite que un equipo pequeño mantenga el código organizado sin la complejidad ni la infraestructura que exigen los microservicios.

2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Permiten escalar cada servicio de forma independiente, desplegar cambios sin detener toda la aplicación, usar tecnologías distintas en cada servicio, aislar fallos para que un error no tumbe todo el sistema y que varios equipos trabajen en paralelo.

3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

Porque separa claramente las responsabilidades, lo soportan casi todos los frameworks web modernos, facilita el mantenimiento y el trabajo en equipo, y logra un buen equilibrio entre simplicidad y organización.

4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

Se generan costos y tiempos de desarrollo innecesarios, mayor dificultad de mantenimiento, necesidad de infraestructura cara y sobre-ingeniería; es decir, se complica el proyecto sin obtener un beneficio real.

5. ¿Crees que existe un patrón mejor que todos los demás? Justifica tu respuesta.

No existe un patrón mejor en términos absolutos. Cada uno resuelve problemas distintos y el más adecuado depende del contexto: el tamaño del proyecto, el equipo disponible, la escalabilidad requerida y el presupuesto. La clave está en elegir el que mejor se ajuste a las necesidades del sistema.

Caso práctico

Sistema seleccionado: Ecommerce (tienda en línea)

Para una plataforma de comercio electrónico utilizaría una arquitectura de microservicios (apoyada internamente en MVC dentro de cada servicio). La justificación, según los criterios solicitados, es la siguiente:

- Número de usuarios: un ecommerce puede tener muchos usuarios con picos altos durante ofertas o temporadas, por lo que necesita una arquitectura capaz de crecer bajo demanda.
- Escalabilidad: los microservicios permiten escalar solo los módulos que más se usan (catálogo, carrito, pagos) sin escalar todo el sistema.
- Complejidad: la complejidad es alta, pero se justifica por el tamaño y la cantidad de funciones del sistema.
- Mantenimiento: cada servicio se actualiza o corrige por separado, sin detener toda la tienda.
- Costos: implica mayor inversión en infraestructura, pero resulta rentable cuando el volumen de ventas y usuarios es alto.
- Tamaño del equipo de desarrollo: requiere un equipo mediano o grande que pueda dividirse para trabajar en distintos servicios al mismo tiempo.

Nota: si el proyecto fuera muy pequeño o apenas iniciara, sería más conveniente comenzar con una arquitectura monolítica o MVC y migrar a microservicios solo cuando el crecimiento lo justifique.

Conclusión

A lo largo de esta actividad aprendí que los patrones de arquitectura de software son soluciones probadas que ayudan a organizar un sistema, separar responsabilidades y facilitar su mantenimiento y crecimiento. Cada patrón tiene un propósito distinto y resulta

más útil en ciertos contextos: MVC y la arquitectura en capas aportan orden, la monolítica es ideal para empezar, los microservicios destacan en sistemas grandes y el modelo cliente-servidor es la base de muchas aplicaciones en red.

El patrón que me pareció más interesante fue el de microservicios, porque permite que sistemas enormes funcionen de forma flexible y escalable, aunque también comprendí que su complejidad no siempre se justifica.

La arquitectura influye directamente en la calidad de un sistema, ya que determina qué tan fácil será mantenerlo, escalarlo, corregir errores y agregar nuevas funciones. Una buena decisión arquitectónica desde el inicio del proyecto evita problemas de rendimiento y costos elevados a futuro, mientras que una mala elección puede comprometer todo el desarrollo. Por eso es fundamental elegir la arquitectura adecuada según las necesidades reales de cada proyecto.